

Comprehensive Guide to Building Software with AI Blueprint

Welcome to the structured era of AI-assisted development. "Vibe coding"—where you let an AI blindly write chunks of application logic through endless chat prompts—is fast at first, but it quickly breaks down into unmaintainable, messy code.

AI Blueprint solves this by acting as a file-backed, spec-driven workflow framework. It forces AI agents (like Claude Code, Codex, GitHub Copilot, or OpenCode) to plan, build, verify, and review one feature at a time. The state of your project lives in readable Markdown files, not in a disappearing chat window.

Crucially, AI Blueprint is completely stack-agnostic. It works perfectly with Node.js, Python (Django, FastAPI), Ruby on Rails, Go, or any other language or framework you choose. It manages the *workflow*, not the application code.

This document is your step-by-step manual to using AI Blueprint flawlessly from day one, regardless of your tech stack.

1. Core Philosophy: Why Are We Doing This?

Before diving into the commands, you must understand the rules AI Blueprint enforces to keep your project perfect:

- **Spec before code:** The AI must write a specification for a feature and wait for your approval before writing any code.
- **One work item at a time:** You only build exactly what is currently planned.
- **Proof before completion:** A green build doesn't mean it works. You must test behavior against the spec before marking it complete.
- **Durable State:** All plans, specs, and history are saved as .md files in your repo. If you clear your AI chat, the AI can instantly pick up where it left off by reading those files.
- **Language Independence:** The framework wraps around your chosen language environment. Your stack, architecture, and product choices remain yours.

2. Phase 1: Setup and Onboarding (Step-by-Step)

Do not install AI Blueprint into an empty folder. It is a workflow framework, not a code boilerplate. You must scaffold your actual application first.

(Note: AI Blueprint itself is a Node CLI tool, so you must have Node.js 22+ installed on your machine to run the initialization and dashboard commands, even if you are building a Python app).

Step 1: Scaffold your app

First, create your application using your preferred stack and initialize Git.

Example A: Python (Django)

```
# Set up your Python environment first
python -m venv venv
source venv/bin/activate
pip install django

# Scaffold the Django project
django-admin startproject my_django_app .
git init
```

Example B: Node.js (Next.js)

```
npx create-next-app@latest my-app
cd my-app
git init
```

Step 2: Install AI Blueprint

Run the installer at the root of your newly scaffolded app. Because Blueprint is distributed via npm, you use npx to install it into your project folder.

```
npx create-ai-blueprint@latest
```

(The interactive installer will ask you which AI adapter you are using, e.g., Claude Code, Codex, GitHub Copilot, or OpenCode. It will add workflow files but leave your application files, like README.md, alone).

Step 3: Run the Onboard Command

Open your AI coding chat (e.g., Claude Code terminal, Cursor chat, or Copilot interface) and use your first command.

(Note: The exact syntax depends on your tool. For Claude Code use /onboard, for Codex use \$onboard, for Copilot just ask it to run the onboard skill).

/onboard

- **Why use this?** This is critical for non-Node stacks. This teaches Blueprint about your real stack (e.g., "This is a Django app running in a virtual environment"), your testing commands

(python manage.py test), conventions, and adapter setup.

Step 4: Write Your Plans

You need two files to guide the AI:

1. blueprint/project-plan.md: The high-level product vision, users, and stack details (e.g., "Python 3.12, Django 5, PostgreSQL").
2. blueprint/build-plan.md: A simple markdown checklist of features to build.
(Note: If you don't know how to write these yet, you can run **/discovery** to have the AI help you draft them).

Step 5: Generate Project Context

/overview

- **Why use this?** This command takes your raw plans and generates a durable project-overview.md. It formats your checklist into a tracked roadmap without changing the scope. On your first run, it will also offer a local Git commit for the setup.

3. Phase 2: The Core Development Loop

This is the heartbeat of your project. For every single feature on your build-plan.md, you will follow this exact 5-step loop. Never skip a step.

Step 1: Spec the Feature

/feature

- **What it does:** The AI picks the next item from your build plan and writes a detailed specification (current-feature.md).
- **Why use this?** It forces human control. You must review and approve this spec before the AI is allowed to write a single line of code.

Step 2: Build the Feature

/implement

- **What it does:** The AI writes the code based strictly on the approved spec, usually in small, reviewable code diffs.
- **Why use this?** It prevents the AI from hallucinating extra features or unnecessary dependencies. It implements exactly what was agreed upon.

Step 3: Prove it Works

/check

- **What it does:** It verifies the real behavior against the acceptance criteria defined in the spec. For a Django app, the AI might run your Python tests here.
- **Why use this?** To guarantee quality. (You can also use /check guide if you want the AI to

give you manual testing instructions to verify the UI yourself).

Step 4: Review the Code

/audit current

- **What it does:** The AI reviews the complete branch delta, checking for defects, security risks, or architectural drift.
- **Why use this?** Unresolved severe findings will block the feature from being completed. It acts as an automated Senior Developer PR review.

Step 5: Land the Feature

/complete

- **What it does:** Runs final gates, archives the current feature into history/features/, ticks the box on your build-plan.md, and prepares for a Git merge.
- **Why use this?** It cleans up the workspace and creates a permanent, readable history of what was built, meaning the AI context is perfectly updated for the next feature.

4. The "Slashes": Command Dictionary

Here is a comprehensive breakdown of the core slash commands you will use in your AI chat prompt. *(Remember to adapt the / to your specific AI tool's syntax).*

A. The Build Commands (The Daily Drivers)

- **/feature:** Turns one build-plan item into a spec for your approval.
- **/implement:** Builds the approved spec.
- **/check:** Verify real behavior against the spec. Use /check guide for a read-only manual walkthrough.
- **/complete:** Run final gates, archive the work, and request merge approval.

B. Understand & Review Commands (For Research & Clarity)

- **/explore [topic]:** Investigate an idea against the code without writing files or requiring plans (e.g., /explore using celery for background tasks).
- **/brief [item]:** Explain an existing planned feature, its dependencies, and likely size without writing files.
- **/status:** Show progress, drift, blockers, and the suggested next action.
- **/debug:** Reproduce and diagnose a failure when the cause is unclear.
- **/audit:** Review code for defects and risks.
- **/doctor:** Check if Blueprint setup is correct.

C. Plan & Setup Commands

- **/onboard:** Tune a fresh scaffold to your specific stack and workflow.
- **/adopt:** Survey existing code if you are adding Blueprint to a project that already has

shipped features.

- **/discovery**: Develop project plans through conversation.
- **/overview**: Generate project context.
- **/prototype**: Explore visual direction.
- **/tests**: Set up unit or browser tests.
- **/ci**: Automate project checks.

D. Recover & Release Commands (When Things Go Wrong or Go Live)

- **/fix**: Spec an unplanned change or confirmed bug.
- **/rollback**: Plan a safe reversal of a completed feature without erasing its history.
- **/release**: Prepare deployment readiness (e.g., checking WSGI settings for Python).

E. Automation

- **/autopilot**: Build one bounded work item with configured gates.
- **/continuous**: Complete planned features serially.

5. The Live Local Dashboard

You don't have to guess the state of your project. AI Blueprint includes a live visual dashboard that reads your markdown files.

Open a terminal in your project and run:

```
npx create-ai-blueprint@latest dashboard
```

This opens a read-only local browser view (on 127.0.0.1) showing your roadmap progress, current active work, code audit warnings, and your Git state. Keep this open on a second monitor while you work.

6. Best Practices for Zero Mistakes

To ensure your upcoming project goes perfectly:

1. **Be explicit about your stack during onboarding**: When using Python/Django, ensure your project-plan.md clearly states how to run the server and tests so the AI doesn't guess.
2. **Never skip /feature review**: If the spec looks wrong or too complex, tell the AI to adjust it *before* running /implement.
3. **Keep features small**: Update your build-plan.md to have bite-sized checklists. Break large epics down.
4. **Use /status if you get lost**: If you forget where you are in the loop, just type /status.
5. **Do not edit current-feature.md manually while the AI is working**: Let the commands manipulate the workflow files.
6. **Always commit after /complete**: Lock in your perfectly tested code to Git before moving

on to the next item on the roadmap.